Trackhive

Documento de arquitectura
Modelo C4

Sistema de solicitudes de consulta del archivo del CLENA

Proyecto P4 · Curso Backend, Proyecto Final «Soluciones para el CLENA»

Equipo: Grupo 1

Integrantes: Bryan Montoya, Alan García, Álvaro Jiménez y Samuel Ricardo

Versión 1.0 · 11 de noviembre de 2026

Repositorio: github.com/bryanmg20/Trackhive

Índice

1. Introducción 3
2. Nivel 1: Contexto 4
3. Nivel 2: Contenedores 6
4. Nivel 3: Componentes de la API 8
5. Estructura de carpetas del backend 10
6. Módulo NestJS: unión de un caso de uso con su repositorio 11
7. Decisiones técnicas principales 12
8. Resumen de decisiones 14
9. Decisiones detalladas 16

Lista de figuras

Figura 1. Diagrama de contexto de Trackhive (C4, nivel 1) 4
Figura 2. Diagrama de contenedores de Trackhive (C4, nivel 2) 6
Figura 3. Diagrama de componentes de la API de Trackhive (C4, nivel 3) 8

1. Introducción

Trackhive es el sistema con el que el CLENA recibe las solicitudes de consulta de su archivo, sigue su estado, controla la disponibilidad del material y agenda las citas de consulta.

Este documento describe su arquitectura con el modelo C4, que muestra el sistema en niveles de detalle creciente:

- **Nivel 1, Contexto:** el sistema, las personas que lo usan y los sistemas externos.
- **Nivel 2, Contenedores:** las aplicaciones y almacenes de datos que forman el sistema, con su tecnología y el protocolo de cada conexión.
- **Nivel 3, Componentes:** las piezas internas de la API y sus dependencias.

El nivel 4 (código) no se dibuja: lo cubren la estructura de carpetas (sección 5) y el fragmento del módulo NestJS (sección 6).

Las figuras son imágenes exportadas a `docs/img/`. Cada una incluye su leyenda, y cada relación lleva una etiqueta con lo que hace y, entre corchetes, el protocolo.

2. Nivel 1: Contexto

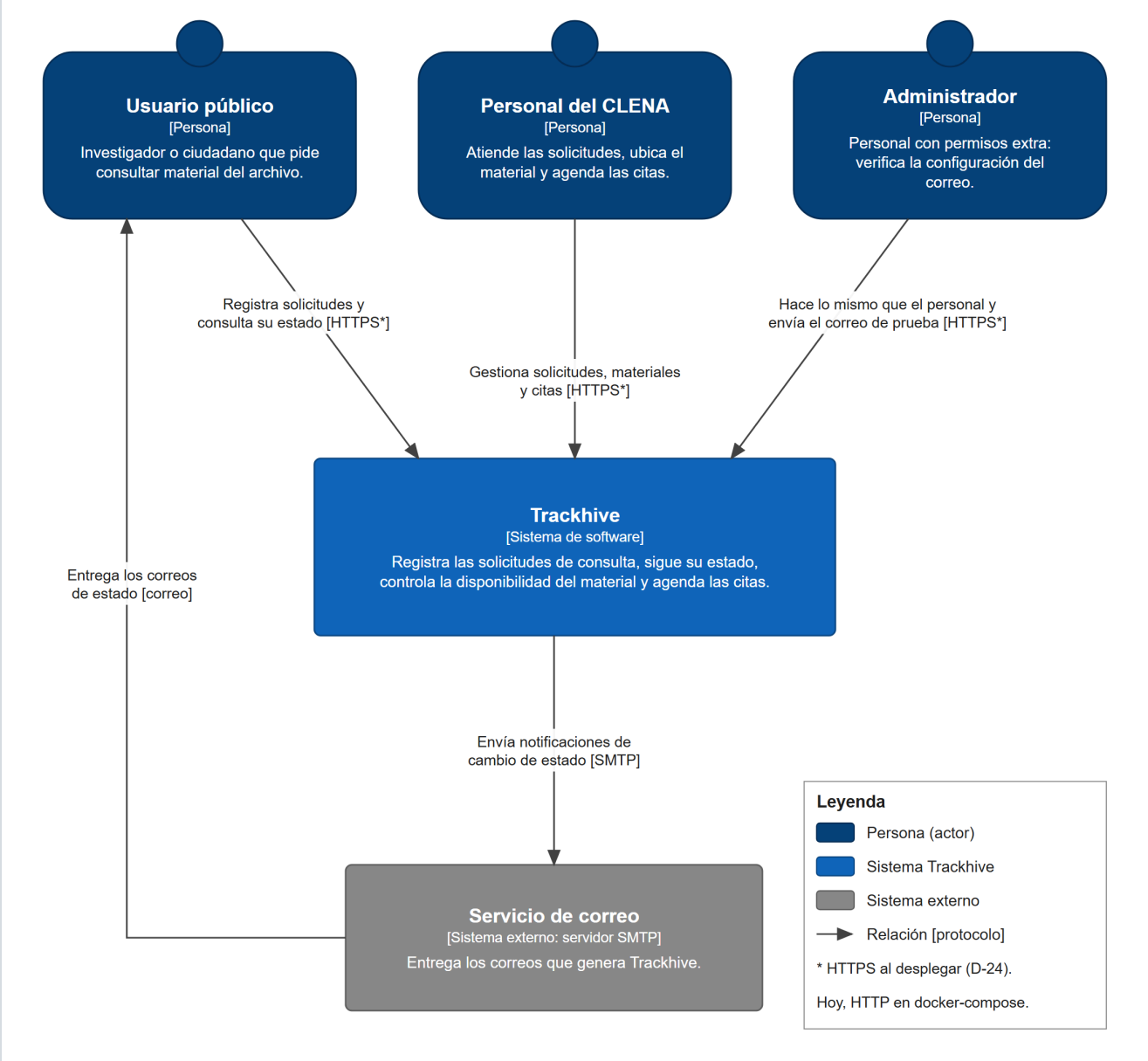


Figura 1. Diagrama de contexto de Trackhive (C4, nivel 1).

Leyenda: azul oscuro, persona; azul, el sistema Trackhive; gris, sistema externo; flecha, relación con su protocolo entre corchetes; el asterisco indica HTTPS al desplegar (hoy HTTP).

Elemento	Tipo	Descripción
Usuario público	Persona (rol `PUBLICO`)	Investigador o ciudadano. Se registra, envía solicitudes de consulta y revisa su estado en «Mis solicitudes».
Personal del CLENA	Persona (rol `PERSONAL`)	Atiende la bandeja de solicitudes, busca el material, cambia estados, registra solicitudes a nombre de un solicitante y agenda las citas.
Administrador	Persona (rol `ADMIN`)	Tiene los permisos del personal y además puede enviar el correo de prueba que verifica la configuración SMTP.

Elemento	Tipo	Descripción
Trackhive	Sistema de software	El sistema que describe este documento.
Servicio de correo	Sistema externo	Servidor SMTP que entrega al solicitante los correos de cambio de estado. En desarrollo es Mailpit, que guarda los correos sin enviarlos.

3. Nivel 2: Contenedores

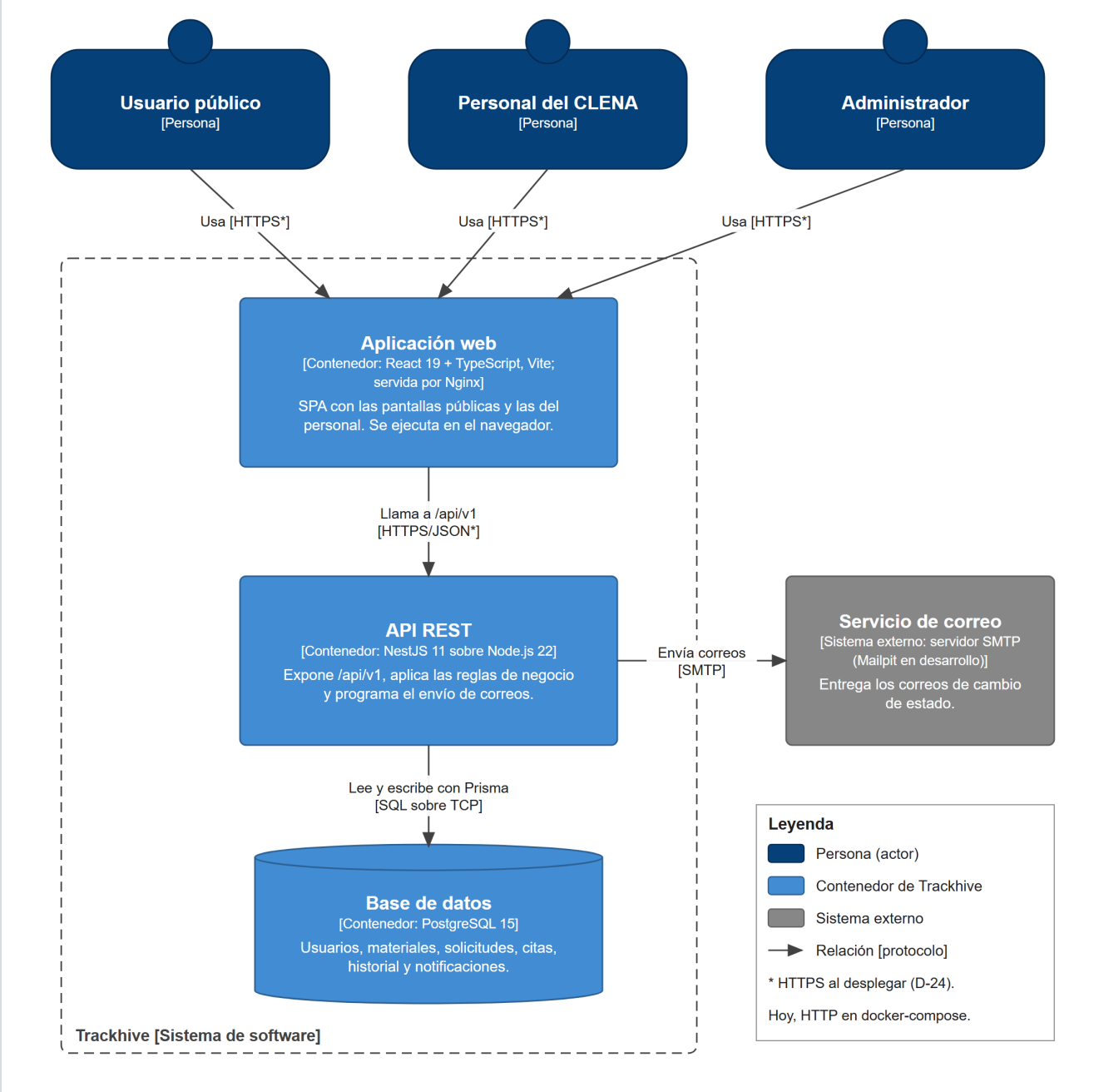


Figura 2. Diagrama de contenedores de Trackhive (C4, nivel 2).

Leyenda: azul oscuro, persona; azul, contenedor de Trackhive; gris, sistema externo; flecha, relación con su protocolo entre corchetes; el asterisco indica HTTPS al desplegar (hoy HTTP).

Contenedor	Tecnología	Responsabilidad
Aplicación web	React 19, TypeScript y Vite. En producción la sirve Nginx 1.27 como archivos estáticos.	SPA que se ejecuta en el navegador: pantallas públicas (registro, inicio de sesión, solicitudes) y pantallas del personal (bandeja, materiales, citas).

Contenedor	Tecnología	Responsabilidad
API REST	NestJS 11 sobre Node.js 22	Expone `/api/v1` y la documentación Swagger en `/docs`. Aplica las reglas de negocio, la autenticación con JWT y la autorización por roles, y programa el envío de correos.
Base de datos	PostgreSQL 15	Guarda usuarios, sesiones, materiales, solicitantes, solicitudes, historial de estados, citas y la tabla de salida de notificaciones.
Servicio de correo	Servidor SMTP (externo). Mailpit 1.32 en desarrollo.	Entrega los correos de cambio de estado.

Conexión	Protocolo	Detalle
Personas → Aplicación web	HTTPS al desplegar; hoy HTTP	El navegador descarga la SPA.
Aplicación web → API REST	HTTPS/JSON al desplegar; hoy HTTP/JSON	Peticiones a `/api/v1` con `Authorization: Bearer <accessToken>`. Nginx reenvía `/api/` y `/docs` a la API.
API REST → Base de datos	SQL sobre TCP (puerto 5432)	Consultas de Prisma Client. Las migraciones se aplican con `prisma migrate deploy`.
API REST → Servicio de correo	SMTP	Nodemailer, configurado con las variables `SMTP_*`.

Trackhive no almacena archivos. Solo gestiona las solicitudes: las copias digitales del material las entrega el personal del CLENA fuera del sistema, por correo.

Estado actual de HTTPS: hoy todas las conexiones del navegador usan HTTP. Nginx solo escucha en el puerto 80 y reenvía a la API con `proxy_pass http://backend:3000`. Las figuras marcan `HTTPS*` porque es el protocolo previsto: el certificado TLS lo pondrá la plataforma de despliegue, que sigue pendiente (D-24).

4. Nivel 3: Componentes de la API

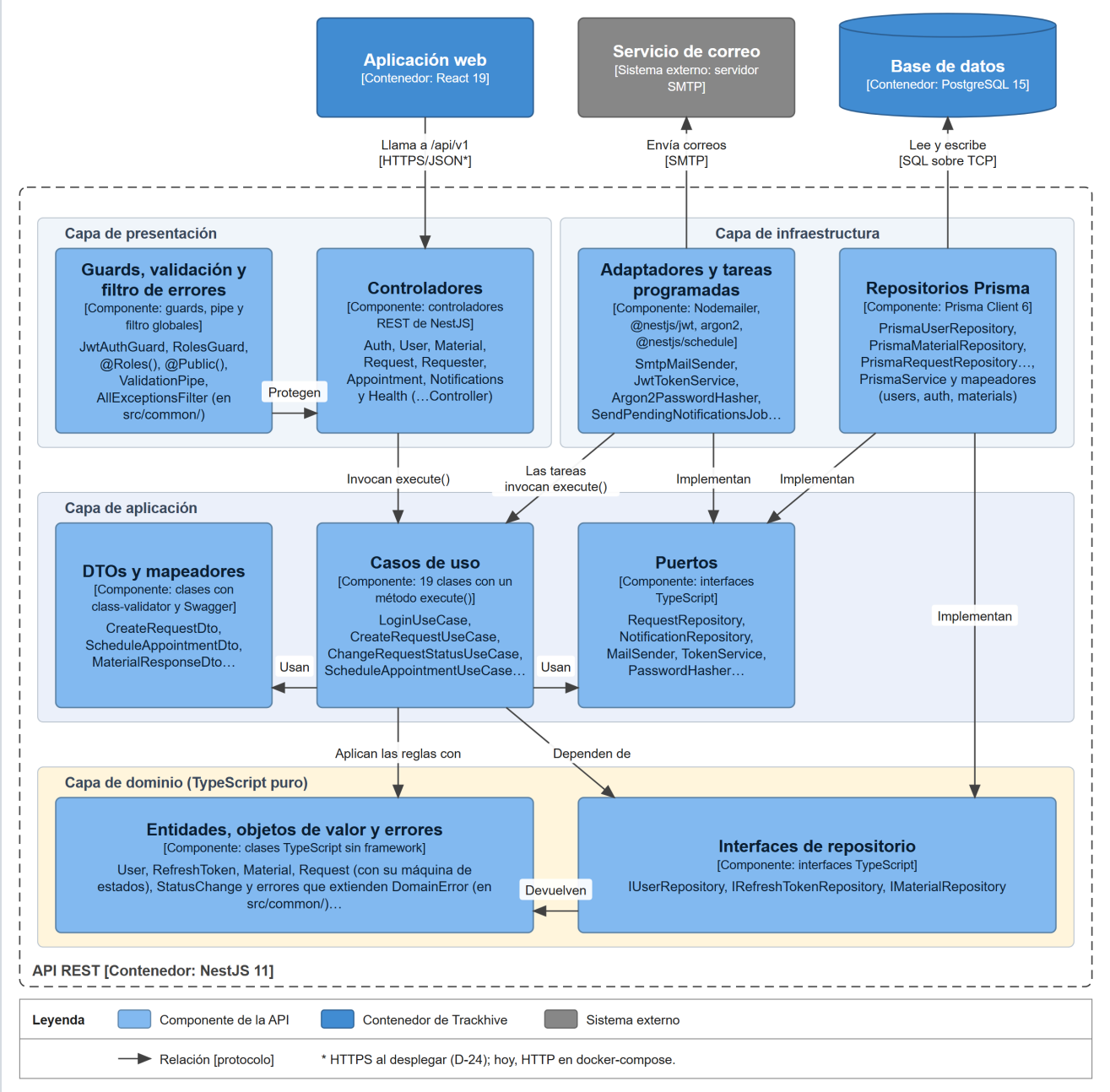


Figura 3. Diagrama de componentes de la API de Trackhive (C4, nivel 3).

Leyenda: azul claro, componente de la API; azul, contenedor de Trackhive; gris, sistema externo; flecha, dependencia o relación con su protocolo; el asterisco indica HTTPS al desplegar (hoy HTTP). Las franjas son las cuatro capas: presentación e infraestructura arriba, aplicación en medio y dominio abajo, de modo que todas las dependencias bajan hacia el dominio. La fila superior, fuera de la API, reúne lo que se comunica con ella.

Regla de dependencia: las dependencias apuntan hacia el dominio. El dominio no importa NestJS, Prisma ni librerías de infraestructura. Infraestructura implementa las interfaces que declaran el dominio y la aplicación.

Capa	Componentes	Contiene	No puede importar
Presentación	Controladores; guards, validación y filtro de errores	Ocho controladores bajo `/api/v1`, `JwtAuthGuard` y `RolesGuard` globales, `@Roles()`, `@Public()`, `ValidationPipe` global y `AllExceptionsFilter` global	Repositorios
Aplicación	Casos de uso; DTOs y mapeadores; puertos	19 casos de uso con un método `execute`, DTOs de petición y de respuesta, puertos (`MailSender`, `TokenService`, `PasswordHasher`, `RequestRepository`…) y los tokens de inyección	Prisma, controladores
Dominio	Entidades, objetos de valor y errores; interfaces de repositorio	`User`, `RefreshToken`, `Material`, `Request` (con su máquina de estados), `StatusChange`, errores que extienden `DomainError`, `IUserRepository`, `IRefreshTokenRepository`, `IMaterialRepository`	NestJS, Prisma, librerías externas
Infraestructura	Repositorios Prisma; adaptadores y tareas programadas	`Prisma…Repository`, mapeadores de `users`, `auth` y `materials`, `SmtpMailSender`, `JwtTokenService`, `Sha256TokenHasher`, `Argon2PasswordHasher`, `SendPendingNotificationsJob`, `PurgeExpiredTokensJob`	Controladores

Tres precisiones sobre el código actual:

- `src/common/` es una carpeta de código compartido, fuera de las cuatro capas. La figura 3 no la dibuja aparte: muestra `AllExceptionsFilter` en presentación y `DomainError` en dominio, ambos con la nota «en `src/common/`». Contiene `AllExceptionsFilter`, `validationExceptionFactory`, `setupSwagger` y la clase base `DomainError`, que no importa ningún paquete externo. Los errores del dominio la extienden, y esa es la única dependencia del dominio hacia fuera de su capa.
- Las interfaces de repositorio de `users`, `auth` y `materials` están en `domain/repositories/`. Las de `requests` y `notifications` (`RequestRepository`, `NotificationRepository`) están en `application/ports/`, porque devuelven modelos de lectura propios de los casos de uso. En ambos casos la implementación Prisma depende de la interfaz, nunca al revés.
- Los casos de uso usan los decoradores `@Injectable()` e `@Inject()` de NestJS para la inyección de dependencias (D-04). El dominio no importa ningún paquete externo.

5. Estructura de carpetas del backend

El backend se organiza primero por módulo funcional y, dentro de cada módulo, por las cuatro capas. Esta es la estructura real de `backend/`:

```
backend/
├── prisma/
│   ├── schema.prisma            # modelos de persistencia
│   ├── migrations/              # migraciones SQL versionadas
│   └── seed.ts                  # datos de demostración
├── src/
│   ├── main.ts                  # arranque de la aplicación
│   ├── app.module.ts            # módulo raíz: importa los módulos funcionales
│   ├── app.setup.ts             # prefijo /api/v1, ValidationPipe, filtro y Swagger
│   ├── common/                  # código compartido por todos los módulos
│   │   ├── errors/              # DomainError y formato de error
│   │   ├── filters/             # AllExceptionsFilter
│   │   ├── html/  swagger/  validation/
│   ├── prisma/                  # PrismaModule (global) y PrismaService
│   ├── auth/
│   │   ├── auth.module.ts
│   │   ├── domain/              # entities/  errors/  repositories/
│   │   ├── application/         # use-cases/  dto/  ports/  services/  tokens.ts
│   │   ├── infrastructure/      # persistence/  mappers/  security/  jobs/
│   │   └── presentation/        # controllers/  guards/  decorators/
│   ├── users/
│   │   ├── users.module.ts
│   │   ├── domain/              # entities/  errors/  repositories/
│   │   ├── application/         # use-cases/  dto/  mappers/  ports/  tokens.ts
│   │   ├── infrastructure/      # persistence/  mappers/  security/
│   │   └── presentation/        # controllers/
│   ├── materials/
│   │   ├── materials.module.ts
│   │   ├── domain/              # entities/  errors/  repositories/
│   │   ├── application/         # use-cases/  dto/  mappers/  tokens.ts
│   │   ├── infrastructure/      # persistence/  mappers/
│   │   └── presentation/        # controllers/
│   ├── requests/
│   │   ├── requests.module.ts
│   │   ├── domain/              # entities/  errors/  y objetos de valor
│   │   ├── application/         # use-cases/  dto/  mappers/  ports/  notifications/  tokens.ts
│   │   ├── infrastructure/      # persistence/
│   │   └── presentation/        # controllers/
│   ├── notifications/
│   │   ├── notifications.module.ts
│   │   ├── domain/              # errors/
│   │   ├── application/         # use-cases/  dto/  ports/  tokens.ts
│   │   ├── infrastructure/      # persistence/  mail/  jobs/
│   │   └── presentation/        # controllers/
│   └── health/
│       ├── health.module.ts
│       ├── application/         # use-cases/  dto/  ports/  tokens.ts
│       ├── infrastructure/      # database/
│       └── presentation/        # controllers/
└── test/                        # pruebas e2e y fakes en memoria
```

Las pruebas unitarias (`*.spec.ts`) están junto al archivo que prueban.

6. Módulo NestJS: unión de un caso de uso con su repositorio

El módulo `materials` muestra cómo se cumple la regla de dependencia con la inyección por tokens (D-04).

El token se declara en la capa de aplicación (`materials/application/tokens.ts`):

```
export const MATERIAL_REPOSITORY = Symbol('MATERIAL_REPOSITORY');
```

El caso de uso pide la interfaz del dominio, no la clase de Prisma (`materials/application/use-cases/search-materials.use-case.ts`):

```
@Injectable()
export class SearchMaterialsUseCase {
  constructor(
    @Inject(MATERIAL_REPOSITORY)
    private readonly materialRepo: IMaterialRepository,
  ) {}
  // ...
}
```

El módulo es el único lugar que conoce la implementación y la enlaza con el token (`materials/materials.module.ts`):

```
@Module({
  controllers: [MaterialController],
  providers: [
    {
      provide: MATERIAL_REPOSITORY,
      useClass: PrismaMaterialRepository,
    },
    SearchMaterialsUseCase,
    ChangeMaterialStatusUseCase,
  ],
})
export class MaterialsModule {}
```

Con esto, `SearchMaterialsUseCase` depende solo de `IMaterialRepository`. En las pruebas unitarias se le entrega un repositorio fake en memoria, y cambiar de ORM solo exige otra clase en `useClass`.

7. Decisiones técnicas principales

Esta sección justifica las cuatro decisiones que más condicionan el código. Las demás están en las secciones 8 y 9.

7.1 ORM: Prisma

- **Decisión:** Prisma 6 (Prisma Client y Prisma Migrate), confinado a la capa de infraestructura.
- **Alternativas:** TypeORM.
- **Justificación:** genera tipos a partir de `schema.prisma` y migraciones SQL versionadas que se revisan en cada PR. Las reglas que el esquema no expresa (`CHECK`, triggers del historial) se escriben a mano en el SQL de la migración.
- **Consecuencias:** las entidades de dominio no son los modelos de Prisma; cada repositorio convierte entre ambos (D-05). El detalle está en D-06.

7.2 Validación: class-validator en la API y Zod en el frontend

- **Decisión:** en la API, los DTOs de petición (`application/dto/`) usan decoradores de `class-validator` y `class-transformer`, y un `ValidationPipe` global los aplica con `whitelist`, `forbidNonWhitelisted` y `transform`. Las reglas de negocio se validan en las entidades y los casos de uso. En el frontend, los formularios de registro e inicio de sesión usan React Hook Form con esquemas de Zod. Los demás (nueva solicitud, cambio de estado y agendar cita) manejan su estado con `useState` y validan a mano los campos obligatorios.
- **Alternativas:** validar a mano en cada controlador; usar Zod también en la API.
- **Justificación:** `class-validator` es la opción que NestJS integra de fábrica y comparte los DTOs con Swagger. El pipe global garantiza que ningún endpoint reciba campos no declarados. Separar forma (DTO) de reglas (dominio) mantiene el dominio sin decoradores de framework (D-13).
- **Consecuencias:** los mensajes de validación se devuelven en un formato de error uniforme, armado por `validationExceptionFactory` y `AllExceptionsFilter` (D-14). Una regla de forma puede existir dos veces, en el esquema Zod y en el DTO; la API es la que manda.

7.3 Logger: el `Logger` de NestJS

- **Decisión:** se usa el `Logger` que NestJS trae por defecto en `@nestjs/common`, sin configuración propia, con una instancia por clase (`new Logger(Clase.name)`), en el filtro de errores, las tareas programadas, el envío de correo y los casos de uso que cambian estados.
- **Alternativas:** `nestjs-pino`; Winston.
- **Justificación:** es el logger por defecto: no agrega dependencias, marca cada línea con el nombre de la clase y permite niveles (`log`, `warn`, `error`). Para el volumen del CLENA basta con la salida estándar del contenedor, que se consulta con `docker compose logs`.
- **Consecuencias:** hoy los logs salen como texto. El curso pide logs estructurados para la entrega final; la salida en JSON sigue pendiente (D-23). Los mensajes no incluyen datos personales: por ejemplo, un correo fallido se registra con el id de la notificación y el radicado, sin el destinatario.

7.4 Librería de componentes del frontend: ninguna, componentes propios con CSS Modules

- **Decisión:** el frontend no usa una librería de componentes. Los componentes son propios (por ejemplo, `TextField` y `AuthLayout`), con estilos en CSS Modules y una hoja global (`src/styles/global.css`). Las fuentes se empaquetan con `@fontsource`.
- **Alternativas:** Material UI; Chakra UI; Tailwind con shadcn/ui.
- **Justificación:** la aplicación tiene pocas pantallas y pocos tipos de control. Con componentes propios el equipo controla el marcado y cumple los criterios de accesibilidad de HU-30 (etiquetas, foco, contraste), que se verifican con `eslint-plugin-jsx-a11y` y pruebas sobre la hoja global. Además, evita el peso y las reglas de estilo de una librería grande.
- **Consecuencias:** cada control nuevo (tablas, diálogos, calendario) se construye y se prueba a mano. La vista de calendario de la agenda también es un componente propio (D-20).

8. Resumen de decisiones

La columna «Estado» se revisó contra el código el 11 de noviembre de 2026. «Parcial» indica entre paréntesis lo que falta.

ID	Decisión	Estado
D-01	Clean Architecture con regla de dependencia estricta	Implementada
D-02	Monorepo con `backend/`, `frontend/` y `docs/`	Implementada
D-03	Un caso de uso por clase, con método `execute`	Implementada
D-04	Inyección de dependencias por tokens	Implementada
D-05	Entidades de persistencia separadas de las de dominio, con mapeadores	Implementada
D-06	Prisma como ORM	Implementada
D-07	La máquina de estados vive en la entidad `Request`	Implementada
D-08	Historial de estados en tabla propia, solo de inserción	Implementada
D-09	Sin dobles reservas: un material por día, con índice único en PostgreSQL	Implementada
D-10	Notificaciones por puerto, con tabla de salida y reintentos	Implementada
D-11	JWT (access y refresh) con rotación y contraseñas con argon2	Implementada
D-12	Autorización por roles con guards globales; propiedad de datos en casos de uso	Implementada
D-13	Validación en dos niveles: forma en presentación, reglas en dominio	Implementada
D-14	Errores de dominio traducidos por un filtro global	Implementada
D-15	API REST versionada con OpenAPI; paginación por página y límite	Implementada
D-16	Búsqueda de material con `pg_trgm`, sin motor de búsqueda externo	Implementada
D-17	Sin integración automática con *Everything* ni lectura de buzón IMAP	Vigente (integración con *Everything* posible como mejora futura)

ID	Decisión	Estado
D-18	Fechas en UTC (`timestamptz`) y presentación en `America/Bogota`	Implementada
D-19	Cumplimiento de la Ley 1581: consentimiento versionado y anonimización	Parcial (faltan eliminar la cuenta, HU-33, y la aceptación del solicitante que registra el personal). **Validar con el CLENA**
D-20	Frontend con Vite, React Router y React Hook Form con Zod; sesión con access y refresh token	Implementada
D-21	Pruebas: fakes en memoria para unitarias y PostgreSQL real para e2e	Parcial (e2e aún con fakes; PostgreSQL real es meta del equipo, HU-41)
D-22	Docker, docker-compose y GitHub Actions	Implementada
D-23	Configuración por entorno validada al arrancar y logs estructurados	Parcial (faltan los logs en JSON)
D-24	Plataforma de despliegue	**Pendiente**

9. Decisiones detalladas

D-01. Clean Architecture con regla de dependencia estricta

- **Decisión:** el backend se organiza en dominio, aplicación, infraestructura y presentación; las dependencias apuntan siempre hacia el dominio.
- **Alternativas:** arquitectura en capas clásica de NestJS (controlador → servicio → repositorio); modular por funcionalidad sin capas explícitas.
- **Justificación:** es un requisito del curso y se evalúa de forma directa. Además aporta valor real: las reglas de estados y de agenda son lo más delicado del sistema y quedan aisladas del framework y de la base de datos, de modo que se prueban sin levantar nada.
- **Consecuencias:** más archivos y más mapeos que en un CRUD directo. A cambio, el CLENA podría cambiar la base de datos o el proveedor de correo sin tocar las reglas de negocio.

D-02. Monorepo con `backend/`, `frontend/` y `docs/`

- **Decisión:** un único repositorio de GitHub.
- **Alternativas:** dos repositorios independientes.
- **Justificación:** las reglas de evidencia de Scrum se verifican sobre un solo historial de commits, issues y PRs. Un monorepo facilita que un mismo PR cubra backend y frontend de una historia de usuario, y que un único `docker-compose` levante todo.
- **Consecuencias:** el pipeline de CI debe ejecutar tareas distintas por carpeta; se resuelve con jobs separados.

D-03. Un caso de uso por clase, con método `execute`

- **Decisión:** cada acción de negocio es una clase (`CreateRequestUseCase`, `ScheduleAppointmentUseCase`, `ChangeRequestStatusUseCase`...), con un único método público, `execute`. Los nombres del código van en inglés, que es la convención habitual; los textos que ve el usuario van en español.
- **Alternativas:** servicios grandes agrupados por entidad (por ejemplo, un `RequestsService`).
- **Justificación:** el enunciado pide "casos de uso claros", y las clases pequeñas son fáciles de probar, de revisar en un PR y de asignar entre integrantes sin conflictos.
- **Consecuencias:** muchas clases pequeñas. Se mitiga con una carpeta `application/use-cases/` dentro de cada módulo funcional.

D-04. Inyección de dependencias por tokens

- **Decisión:** las interfaces de repositorio y los puertos se identifican con tokens (un `Symbol` exportado desde el archivo `application/tokens.ts` de cada módulo). El módulo de NestJS enlaza cada token con su implementación de infraestructura.
- **Alternativas:** inyectar clases concretas; usar clases abstractas como tokens.
- **Justificación:** TypeScript borra las interfaces en tiempo de ejecución, así que se necesita un token para que NestJS pueda resolverlas. Con tokens el caso de uso depende solo de la abstracción, y en las pruebas se inyecta un fake sin tocar el contenedor real.

- **Consecuencias:** el enlace vive en un solo lugar (los módulos). Un token olvidado falla al arrancar, no en producción.

D-05. Entidades de persistencia separadas de las de dominio, con mapeadores

- **Decisión:** las entidades del ORM viven en infraestructura y nunca salen de ella; los repositorios las convierten a entidades de dominio. En `users`, `auth` y `materials` lo hace una clase mapeadora (`infrastructure/mappers/`); en `requests` y `notifications` la conversión está dentro del repositorio.
- **Alternativas:** usar una sola clase que sea a la vez entidad de dominio y del ORM.
- **Justificación:** una clase única obligaría al dominio a importar decoradores del ORM, lo que rompe la regla de dependencia (y es motivo de descalificación del criterio de Arquitectura). Separarlas también permite que el modelo de tablas evolucione sin arrastrar a las reglas de negocio.
- **Consecuencias:** código de mapeo adicional que se cubre con pruebas. Es el precio de un dominio puro.

D-06. Prisma como ORM

- **Decisión:** Prisma con migraciones en SQL versionadas (Prisma Migrate).
- **Alternativas:** TypeORM.
- **Justificación:** es el ORM con el que ya se construyó el módulo base, ofrece buena ayuda de tipos y genera las migraciones a partir de `schema.prisma`. Su esquema no expresa los `CHECK`, el trigger del historial (D-08) ni la extensión `pg_trgm`, pero eso se agrega a mano en el SQL de la migración, creada con `prisma migrate dev --create-only`. Los índices de trigramas (D-16) y el índice único de la agenda (D-09) sí quedan declarados en `schema.prisma`.
- **Consecuencias:** Prisma no tiene `synchronize`; su equivalente es `prisma db push`, que no se usa fuera del desarrollo local (requisito del curso). Fuera de local las migraciones se aplican con `prisma migrate deploy`. Hay que revisar las migraciones generadas antes de fusionarlas. La decisión es reversible porque el ORM queda confinado a infraestructura.

D-07. La máquina de estados vive en la entidad `Request`

- **Decisión:** las transiciones son las de `docs/modelo-datos.md` (§1.3): `RECIBIDA → EN_BUSQUEDA | RECHAZADA`; `EN_BUSQUEDA → DISPONIBLE | RECHAZADA`; `DISPONIBLE → AGENDADA (presencial) | ATENDIDA (digital) | CERRADA`; `AGENDADA → ATENDIDA | DISPONIBLE`; `ATENDIDA → CERRADA`. `RECHAZADA` y `CERRADA` son estados finales. Se implementan como métodos de la entidad `Request` (`startSearch()`, `markAvailable()`, `schedule()`, `attend()`, `close()`, `reject(reason)`), que lanzan `InvalidStatusTransitionError` (HTTP 409) ante un cambio no permitido. `markAvailable()` cubre las dos acciones de §1.3 que llegan a `DISPONIBLE` (`marcarDisponible` y `volverADisponible`).
- **Alternativas:** validar las transiciones en el controlador, en el caso de uso o con una tabla de reglas en la base de datos.
- **Justificación:** es la regla de negocio central y no debe poder eludirse desde ninguna otra capa. En el dominio se prueba con pruebas unitarias rápidas y sin infraestructura. Rechazar exige un motivo, y se modela como parámetro obligatorio del método.
- **Consecuencias:** cualquier cambio de estado pasa por la entidad. Las reglas exactas se validan con el equipo de Introducción en la fase de Inception.

D-08. Historial de estados en tabla propia, solo de inserción

- **Decisión:** cada transición inserta una fila en `status_history` (estado anterior, nuevo, usuario, fecha, observación). Nunca se actualiza ni se borra.
- **Alternativas:** guardar solo el estado actual; guardar las fechas de cada estado como columnas de `requests`.
- **Justificación:** el requisito de reportes (tiempos de atención) solo es calculable si se conoce cuándo ocurrió cada transición. Además da trazabilidad: se puede responder quién cambió qué y cuándo, algo valioso para un archivo patrimonial.
- **Consecuencias:** la inserción del historial y la actualización de la solicitud ocurren en una misma transacción, coordinada desde el repositorio.
- **Implementación (HU-15):** `PATCH /api/v1/requests/:id/status` (`@Roles('PERSONAL', 'ADMIN')`) llama a `ChangeRequestStatusUseCase`. La entidad `Request.changeStatus()` aplica la transición con los mismos métodos de D-07 y devuelve un `StatusChange` inmutable (estado anterior, nuevo, responsable, fecha y comentario opcional). `PrismaRequestRepository.saveStatusChange()` actualiza la solicitud solo si sigue en el estado anterior (bloqueo optimista; si otra persona la cambió primero responde 409 `REQUEST_STATUS_CONFLICT`), incrementa `version`, llena `attended_at` o `closed_at` cuando corresponde e inserta la fila de `status_history`, todo en una transacción. Un trigger de la migración rechaza `UPDATE`, `DELETE` y `TRUNCATE` sobre la tabla. El historial se muestra en HU-17.
- **Rechazo con motivo (HU-16):** el mismo endpoint acepta `RECHAZADA`. `changeStatus()` llama a `reject()`, así que el comentario pasa a ser el motivo y es obligatorio: sin él responde 400 `REJECTION_REASON_REQUIRED` y no se guarda nada. El motivo queda en `requests.rejection_reason` (ampliada a `varchar(500)`, con un CHECK que exige que no esté vacío) y como comentario de la fila de `status_history`. `GET /api/v1/requests/mine` lo devuelve para que el solicitante lo vea en «Mis solicitudes».
- **Implementación (HU-18):** `AGENDADA` ya no se acepta en `PATCH /status`: solo se alcanza con `POST /api/v1/requests/:id/appointments` (`@Roles('PERSONAL', 'ADMIN')`), que llama a `ScheduleAppointmentUseCase`. `Request.scheduleAppointment()` pasa por `schedule()` (solo `DISPONIBLE` y presencial) y devuelve el `StatusChange`. `PrismaRequestRepository.saveAppointment()` guarda, en una transacción, el cambio de estado con el mismo bloqueo optimista, su fila de `status_history` y la cita del día con el solicitante y todos los materiales de la solicitud. La fecha se valida en `toAppointmentDay()`: un domingo responde 400 `INVALID_APPOINTMENT_DATE`. Al salir de `AGENDADA` por «asistió» o «no asistió», `saveStatusChange()` marca la cita `CUMPLIDA` o `NO_ASISTIO`.

D-09. Sin dobles reservas de material por día

- **Decisión:** la reserva es por día completo, material y fecha, sin franjas horarias. Mientras la cita está `PROGRAMADA`, `appointment_materials` copia el día en `booked_date`; al cumplirse, registrar inasistencia o cancelarse pasa a `NULL` y el día queda libre. El índice único `(material_id, booked_date)` impide reservar el mismo material dos veces el mismo día — un `NULL` nunca choca, así que solo la cita activa retiene el día. Además, el caso de uso `ScheduleAppointmentUseCase` consulta la disponibilidad antes de insertar: si un material ya tiene cita ese día responde 409 `MATERIAL_ALREADY_BOOKED` nombrando el material, la fecha y el radicado que lo tiene, y si está en restauración o no consultable responde 409 `MATERIAL_NOT_AVAILABLE`; el estado `EN_USO` no bloquea, porque solo aplica al día en que el material está en sala (HU-72). Si la base rechaza la inserción por una carrera (dos funcionarios agendando a la vez), el repositorio traduce la violación del índice a `MaterialAlreadyBookedError`.
- **Alternativas:** solo verificar en el código antes de insertar; franjas horarias con `tstzrange` y una restricción `EXCLUDE` (descartadas: la cita ya no tiene hora, es un día completo, y la igualdad basta sin la extensión `btree_gist`); bloqueos explícitos a nivel de fila.

- **Justificación:** la verificación en código sola es vulnerable a condiciones de carrera. El índice único garantiza la integridad sin importar cuántas instancias de la API corran, y al ser por día no necesita rangos ni extensiones de PostgreSQL. La combinación de verificación previa y restricción es deliberada: la primera da un mensaje claro y la segunda da seguridad.
- **Consecuencias:** la regla vive en una migración SQL y en el dominio (que expresa el concepto), y está documentada en `docs/modelo-datos.md`. `booked_date` se desnormaliza de la cita y se libera en la misma transacción del cambio de estado que saca la solicitud de `AGENDADA` (implementación de HU-20).

D-10. Notificaciones por puerto, con tabla de salida y reintentos

- **Decisión:** `ChangeRequestStatusUseCase` no envía correos: registra una fila en `notifications` (estado `PENDIENTE`) dentro de la misma transacción del cambio de estado. Un proceso programado (`@nestjs/schedule`) la envía a través del puerto `MailSender`, implementado con SMTP (Nodemailer), y marca `ENVIADA` o `FALLIDA` con un contador de reintentos.
- **Alternativas:** enviar el correo de forma síncrona dentro del caso de uso; usar una cola externa (Redis, RabbitMQ).
- **Justificación:** una caída del servidor SMTP no debe impedir ni revertir un cambio de estado legítimo. Una tabla de salida da fiabilidad sin sumar infraestructura nueva, algo importante porque el CLENA debe poder operarlo. Una cola externa se descarta por exceso de complejidad para este volumen.
- **Consecuencias:** el correo no es instantáneo (llega en el siguiente ciclo del proceso, de segundos a un par de minutos). En desarrollo se usa Mailpit para inspeccionar correos sin enviarlos. HU-22 implementó el puerto con el nombre `MailSender` (`notifications/application/ports/`), su versión SMTP con Nodemailer (`SmtpMailSender`, configurada con las variables `SMTP_*`) y el servicio Mailpit en `docker-compose.yml`. Un administrador puede comprobar la configuración con `POST /api/v1/notifications/test-email`, que le envía un correo de prueba a su propio correo.
- **Implementación (HU-23):** todo cambio de estado pasa por `PrismaRequestRepository.applyStatusChange()`, tanto «Cambiar estado» (`PATCH /requests/:id/status`, HU-15 y HU-16) como «Agendar cita» (`POST /requests/:id/appointments`, HU-18). En la misma transacción que actualiza la solicitud y crea la fila de `status_history`, inserta una fila `PENDIENTE` en `notifications` (migración `20261113000000_create_notifications`), enlazada a esa fila del historial. Antes de guardar, `ChangeRequestStatusUseCase` y `ScheduleAppointmentUseCase` arman el correo con `buildStatusChangeEmail()` (`requests/application/notifications/`). El correo lleva el radicado, el nuevo estado y el material, y además la fecha y el horario de la cita en `AGENDADA` o el motivo en `RECHAZADA`. Se guarda en texto plano y en HTML, con los datos escritos por personas escapados. Si el solicitante no tiene correo, el cambio se guarda sin notificación y queda un aviso en el log. `SendPendingNotificationsJob` (`@Interval`, cada 10 s) llama a `SendPendingNotificationsUseCase`. El caso de uso reclama hasta 20 correos con bloqueo optimista sobre `attempts`, así que dos ejecuciones o dos instancias no envían el mismo correo, y deja un plazo de 5 minutos para volver a tomar un correo cuyo envío se cortó. Luego los envía por `MailSender` y los marca `ENVIADA`. Si el envío falla, registra en el log el id del correo, el radicado, el intento y el motivo, sin el destinatario (D-23). El motivo queda en `last_error` y el correo se reintenta a los 1, 5, 15 y 60 minutos; al quinto fallo queda `FALLIDA`. El cambio de estado nunca se revierte por un fallo del correo. La entrega es «al menos una vez»: si la base de datos falla justo después de enviar un correo, ese correo puede repetirse. Las plantillas por estado son de HU-24.

D-11. JWT (access y refresh) con rotación y argon2

- **Decisión:** el access token dura 15 minutos y se firma con `JWT_SECRET`. El refresh token dura 7 días, se firma con `JWT_REFRESH_SECRET` y lleva un `jti` aleatorio para que dos tokens nunca sean iguales. Cada refresh token emitido se guarda en `refresh_tokens` solo como hash SHA-256. `POST /api/v1/auth/refresh` lo rota: revoca el token recibido y entrega un par nuevo. Si llega un token ya revocado (reuso), se revocan todas las sesiones del usuario. `POST /api/v1/auth/logout` revoca el token y responde 204 aunque no exista, como indica el RFC 7009. Las contraseñas se almacenan con argon2. Los tokens viajan en el cuerpo JSON; llevar el refresh token a una cookie `httpOnly` queda como mejora posible (D-20).
- **Alternativas:** sesiones con cookies en servidor; bcrypt; refresh token sin rotación; guardar el refresh token con argon2.
- **Justificación:** JWT con refresh es requisito del curso. La rotación limita el daño de un refresh robado, y guardar solo su hash evita que una filtración de la base entregue sesiones válidas. Para el token basta SHA-256: es largo y aleatorio, así que no se adivina por fuerza bruta como una contraseña, y como el mismo token siempre da el mismo hash, se puede buscar por él. Argon2 usa una sal aleatoria y no permitiría esa búsqueda. Para las contraseñas sí se usa argon2, que resiste mejor ataques por GPU que bcrypt.
- **Consecuencias:** el access token no se puede revocar: sigue sirviendo hasta 15 minutos después de cerrar sesión, y por eso dura poco. La tabla no guarda una familia de tokens, así que ante un reuso se cierran todas las sesiones del usuario y no solo la cadena afectada. Si el frontend lanza dos renovaciones a la vez con el mismo token, la segunda cuenta como reuso, así que debe esperar a que termine la primera. Cada inicio de sesión y cada renovación agregan una fila. Por eso una tarea diaria (`@nestjs/schedule`, a las 3:00 hora de Bogotá, y una vez al arrancar la app) borra las filas vencidas. Las revocadas que aún no vencen se conservan, porque sirven para detectar el reuso (HU-3.1).

D-12. Autorización por roles con guards globales; propiedad en casos de uso

- **Decisión:** `JwtAuthGuard` y `RolesGuard` se registran como guards globales; los endpoints declaran sus roles con `@Roles()` y los públicos con `@Public()`. Reglas como "un usuario público solo ve sus propias solicitudes" se implementan en el caso de uso, que recibe el identificador del usuario.
- **Alternativas:** aplicar guards endpoint por endpoint; resolver la propiedad en el guard.
- **Justificación:** con guards globales ningún endpoint queda desprotegido por olvido. La propiedad de los datos es una regla de negocio, así que no corresponde a presentación.
- **Implementación:** `JwtAuthGuard` valida el `Authorization: Bearer <accessToken>` y deja `{ id, role }` en `request.user`; sin token, o con uno inválido o vencido, responde 401. `RolesGuard` compara ese rol con los de `@Roles()` y responde 403 si no coincide. Ambos están en `backend/src/auth/presentation/`.
- **Consecuencias:** los endpoints sin `@Roles()` exigen al menos autenticación; solo `@Public()` los abre.

D-13. Validación en dos niveles

- **Decisión:** los DTOs de petición (clases con `class-validator` en `application/dto/`) validan forma y tipos. Los aplica en la frontera de presentación un `ValidationPipe` global, con `whitelist` y `forbidNonWhitelisted` activos. Las reglas de negocio (transiciones, fechas coherentes, disponibilidad) se validan en dominio y aplicación.
- **Alternativas:** validar todo en los DTOs; validar todo en el dominio.
- **Justificación:** los decoradores de validación son dependencias de framework, así que no pueden estar en el dominio. Cada nivel responde a una pregunta distinta: "¿la petición está bien formada?" y "¿la operación está permitida?".

- **Consecuencias:** los DTOs viven en la capa de aplicación y llevan decoradores de `class-validator` y de Swagger, así que esa capa depende de esas dos librerías; el dominio no. La mayoría de los casos de uso declaran además su propia entrada sin decoradores (por ejemplo, `SearchMaterialsInput`), y el controlador convierte el DTO en esa entrada.

D-14. Errores de dominio traducidos por un filtro global

- **Decisión:** el dominio y la aplicación lanzan excepciones propias (`InvalidStatusTransitionError`, `MaterialAlreadyBookedError`, `RequestNotFoundError`...). Un filtro global (`AllExceptionsFilter`, en `src/common/filters/`) las traduce a códigos HTTP y a un formato de error uniforme con un `code` estable.
- **Alternativas:** lanzar `HttpException` desde los casos de uso.
- **Justificación:** las `HttpException` pertenecen a NestJS y contaminarían el núcleo. Un `code` estable permite al frontend reaccionar por tipo de error sin depender del texto del mensaje.
- **Consecuencias:** cada error nuevo requiere una línea en el mapeo; un error sin mapear devuelve 500 y se registra en logs.

D-15. API REST versionada con OpenAPI

- **Decisión:** prefijo `/api/v1` con versionado por URI; documentación Swagger en `/docs`; listados con paginación por `page` y `limit`, filtros y ordenamiento.
- **Alternativas:** GraphQL; paginación por cursor.
- **Justificación:** REST con OpenAPI es requisito. La paginación por página es suficiente para el volumen esperado y es más sencilla para una tabla de bandeja con saltos de página. La especificación OpenAPI permite generar el cliente del frontend y evita desajustes de tipos.
- **Consecuencias:** si el volumen crece mucho, habrá que migrar a paginación por cursor; no es una necesidad actual.

D-16. Búsqueda de material con `pg_trgm`

- **Decisión:** la búsqueda es por coincidencia parcial, sin distinguir mayúsculas de minúsculas (`ILIKE '%texto%'`, que Prisma genera con `contains` y `mode: 'insensitive'`). Para que no recorra toda la tabla hay índices GIN con `pg_trgm` sobre el título, el autor y la colección del material y sobre los cuatro campos del nombre del solicitante.
- **Alternativas:** `LIKE` sin índice; Elasticsearch o similar.
- **Justificación:** el personal busca con un fragmento del título, del autor o del nombre, no con el texto exacto. Un índice B-tree normal no sirve cuando el texto buscado puede estar en medio de la columna; un índice de trigramas sí. Así la búsqueda queda dentro de PostgreSQL, sin un servicio extra que el CLENA tendría que operar. Un motor de búsqueda dedicado es desproporcionado para este volumen.
- **Consecuencias:** requiere la extensión `pg_trgm`, que se habilita en la migración que crea `materials`. La búsqueda no corrige errores de escritura: «Garcia» no encuentra «García» ni «Garsia». `pg_trgm` permite agregarlo después con `similarity()` sobre los mismos índices. El código de catálogo, el correo y el documento del solicitante también se buscan por coincidencia parcial, pero sin índice de trigramas. La búsqueda dentro del contenido de los documentos queda fuera de alcance (ver D-17).

D-17. Sin integración automática con *Everything* ni lectura de buzón IMAP

- **Decisión:** el personal busca en *Everything* como hoy y registra el resultado (disponibilidad y ubicación) en Trackhive. Las solicitudes entran por un formulario web público o las registra el personal; no se lee el buzón `labdigital@clena.org` de forma automática.
- **Alternativas:** integrar con *Everything*; procesar correos entrantes por IMAP.
- **Justificación:** la integración con *Everything* es técnicamente posible: el programa incluye un servidor HTTP opcional que responde las búsquedas en JSON (`?search=...&json=1`). No se hace ahora por tres razones. El alcance acordado con el CLENA es que Trackhive solo gestiona las solicitudes, y el enunciado no la exige. La API tendría que alcanzar por red el equipo Windows donde corre *Everything*, y eso depende de la plataforma de despliegue (D-24). Y ese servidor expone los nombres y rutas de los archivos del CLENA, así que habría que activarlo y protegerlo con usuario y contraseña. Leer el buzón añade riesgo (credenciales institucionales, correos con formato libre, datos personales no controlados) para un beneficio que el formulario público ya cubre.
- **Consecuencias:** la integración con *Everything* se documenta como mejora futura: Trackhive podría consultarlo para sugerir al personal la ubicación del material, que hoy se escribe a mano. Antes hay que validarlo con el CLENA. El formulario web es la vía que reduce la lectura manual de correos que describió el personal.

D-18. Fechas en UTC y presentación en `America/Bogota`

- **Decisión:** todas las columnas de fecha y hora son `timestamptz` y se almacenan en UTC; la API devuelve ISO 8601 y el frontend las muestra en `America/Bogota`. Las reglas que dependen de la hora la reciben como parámetro opcional (`now: Date = new Date()`), por ejemplo en `Request.changeStatus()`, `Material.changeStatus()` y `todayInBogota()`.
- **Alternativas:** guardar hora local sin zona; un puerto de reloj (`RelojPort`) inyectado en cada caso de uso.
- **Justificación:** evita errores en reportes y citas ante cambios de servidor o despliegue. El parámetro `now` permite probar con una hora fija las reglas que dependen del tiempo, sin sumar un puerto ni un proveedor más al módulo.
- **Consecuencias:** el frontend es responsable de la conversión; las fechas "solo día" (fecha deseada) se tratan como tipo `date`.

D-19. Cumplimiento de la Ley 1581: consentimiento y anonimización *(validar con el CLENA)*

- **Decisión:** se recogen solo los datos mínimos necesarios; el formulario de registro exige aceptar el aviso de privacidad, y la aceptación se guarda en `users` con fecha y versión del texto. La tabla `requesters` tiene las mismas columnas para el solicitante. La eliminación de cuenta anonimiza los datos personales del solicitante (nombre, correo, teléfono) y conserva las solicitudes y el historial sin datos personales, para no romper los reportes.
- **Alternativas:** borrado físico en cascada.
- **Justificación:** el borrado en cascada destruiría el historial y los indicadores de atención, y sería incompatible con la trazabilidad de un archivo. La anonimización cumple con la supresión del dato personal sin perder la información agregada.
- **Consecuencias:** el equipo debe confirmar con el CLENA si existen obligaciones de conservación que exijan otro tratamiento. No se usan datos reales sin autorización escrita; las semillas son ficticias.

- **Estado actual:** están implementados el aviso de privacidad y el consentimiento versionado al registrarse (HU-31). En `requesters`, la fecha y la versión las escribe el sistema al crear el solicitante: quien tiene cuenta ya aceptó el aviso al registrarse, pero cuando el personal registra a alguien sin cuenta todavía no se recoge su aceptación. Falta también la eliminación de cuenta con anonimización (HU-33), que el curso exige para la entrega final.

D-20. Frontend: Vite, React Router y React Hook Form con Zod; sesión con access y refresh token

- **Decisión:** React con TypeScript sobre Vite. React Hook Form con esquemas de Zod en los formularios de registro e inicio de sesión; los demás formularios usan `useState` (sección 7.2). React Router con rutas protegidas por rol (`ProtectedRoute`, `StaffRoute`). Las llamadas a la API usan `fetch` con una función propia, `authenticatedFetch`, que agrega el access token y, si la API responde 401, renueva la sesión con el refresh token y repite la petición. La vista de calendario de la agenda es un componente propio (`AppointmentsCalendarPage`). Los dos tokens se guardan en `sessionStorage`.
- **Alternativas:** React Query para los datos del servidor; FullCalendar para la agenda; Redux para el estado; guardar los tokens en `localStorage` o llevar el refresh token a una cookie `httpOnly`.
- **Justificación:** la aplicación tiene pocas pantallas y cada una pide sus datos al entrar, así que una función sobre `fetch` basta y evita dependencias. El calendario solo muestra las citas por día, sin franjas horarias (D-09), y por eso no necesita una librería de calendario. `sessionStorage` se borra al cerrar la pestaña, a diferencia de `localStorage`.
- **Consecuencias:** no hay caché de datos entre pantallas: cada pantalla vuelve a consultar la API. `sessionStorage` es legible por JavaScript, así que un ataque XSS podría leer los tokens; lo limitan la corta vida del access token y la rotación del refresh token (D-11). Pasar el refresh token a una cookie `httpOnly` queda como mejora posible: solo cambiaría la capa de presentación de `auth` y la configuración de CORS. La accesibilidad (contraste, etiquetas, teclado, textos escalables) se trata como criterio de revisión en cada PR.

D-21. Pruebas: fakes en memoria para unitarias y PostgreSQL real para e2e

- **Decisión:** las pruebas unitarias del dominio y de los casos de uso usan repositorios fake en memoria. Las pruebas e2e de la API se ejecutan contra un PostgreSQL real (servicio en GitHub Actions y en docker-compose), con migraciones aplicadas.
- **Alternativas:** mocks de librería en todas partes; SQLite en memoria para e2e.
- **Justificación:** los fakes aprovechan que los casos de uso dependen solo de interfaces y mantienen las pruebas rápidas y legibles. Las e2e necesitan PostgreSQL real para aplicar las mismas migraciones de producción: el índice único de la agenda (D-09) y `pg_trgm` (D-16), que SQLite no soporta.
- **Consecuencias:** el CI es algo más lento.
- **Estado actual:** las pruebas unitarias usan fakes en memoria. Las e2e corren en el CI, pero todavía con repositorios en memoria. Pasarlas a un PostgreSQL real es una meta del equipo (HU-41), más estricta que el curso, que pide pruebas e2e de los flujos principales.

D-22. Docker, docker-compose y GitHub Actions

- **Decisión:** `docker-compose up` levanta backend, frontend y PostgreSQL sin pasos manuales (aplica migraciones y semillas de demostración). Cada Pull Request ejecuta lint, pruebas y build en GitHub Actions; `main` queda protegida y requiere CI en verde y una aprobación.
- **Alternativas:** instalación manual en el servidor del CLENA.

- **Justificación:** es requisito y, además, es lo que permite que el CLENA opere el sistema sin el equipo. Un despliegue reproducible reduce el riesgo de que "funcione solo en una máquina".
- **Consecuencias:** `docs/manual-operacion.md` debe cubrir variables de entorno, despliegue y respaldo de la base de datos.

D-23. Configuración por entorno y logs estructurados

- **Decisión:** `@nestjs/config` con validación de las variables al arrancar (la aplicación falla si falta alguna); `.env.example` en el repositorio y ningún secreto en el código. Logs en formato JSON y endpoint `GET /api/v1/health`.
- **Alternativas:** leer `process.env` directamente; logs de texto libre.
- **Justificación:** fallar pronto ante una configuración incompleta evita errores difíciles de rastrear en producción. Los logs estructurados se pueden filtrar y consultar, y `/api/v1/health` permite monitoreo y verificaciones del despliegue.
- **Consecuencias:** hay que evitar registrar datos personales en los logs.
- **Estado actual:** existen `.env.example` y `GET /api/v1/health`, y no hay secretos en el código. Los logs salen como texto con el `Logger` de NestJS (sección 7.3); pasarlos a JSON está pendiente y el curso lo exige para la entrega final. Validar las variables al arrancar es una meta del equipo; hoy `docker-compose` ya se niega a levantar si falta una variable.

D-24. Plataforma de despliegue *(pendiente)*

- **Decisión:** pendiente de definir.
- **Opciones:** un servicio gestionado con contenedores (Render, Railway, Fly.io) o un VPS con docker-compose.
- **Criterios:** costo (idealmente gratuito o muy bajo para la demo), soporte de PostgreSQL 15+, URL pública estable y facilidad para que el CLENA lo mantenga.
- **Consecuencias:** debe resolverse antes del Sprint 3. Cualquier opción debe poder reproducirse a partir del `docker-compose` del repositorio.